# EVENTHUB

## Online Event Management & Ticket Booking System

### Mini Project Report

**Technology:** Flutter, Dart, Supabase, PostgreSQL
**Application Type:** Cross-Platform Event Management & Ticket Booking Application
**Project Name:** EventHub
**Repository:**
VAIBHAV-ops2077/itm-flutter-miniproject-online-event-management-ticket-booking-system-10-09-2026
**Academic Project:** ITM Skills University
**Date:** September 2026

---

# 1. Abstract

EventHub is a cross-platform online event management and ticket booking application developed using Flutter and Supabase. The system is designed to provide a complete digital platform through which event organizers can create and manage events, attendees can discover and book tickets, and administrators can monitor and manage the overall platform.

The application follows a role-based architecture consisting primarily of three user roles: Attendee, Organizer, and Admin. Attendees can browse published events, view event information, select tickets, proceed through checkout, access their booking history, and display QR-based digital tickets. Organizers can create events, configure ticket categories, monitor bookings, view analytics, manage attendees, and perform event-related operations. Administrators provide platform-level supervision through approval and management workflows.

A major technical aspect of EventHub is its use of Supabase and PostgreSQL for backend services. Instead of relying entirely on client-side validation, the architecture places important booking operations inside database functions. The documented `book_ticket` RPC is intended to use PostgreSQL row locking to prevent concurrent users from purchasing the same remaining ticket.

The project also incorporates Riverpod for state management, go_router for navigation, Freezed for immutable data models, QR generation and scanning, analytics visualization, and environment-based configuration.

Overall, EventHub demonstrates the development of a modern Flutter application combined with a cloud-hosted relational backend and addresses important software engineering concepts including authentication, authorization, state management, database design, transaction handling, concurrency control, ticket validation, analytics, and modular application architecture.

---

# 2. Introduction

Event management traditionally involves multiple independent activities such as event creation, ticket distribution, attendee registration, payment processing, ticket validation, and attendance tracking. When these operations are handled manually or through disconnected systems, organizers may face problems such as inaccurate attendance records, duplicate bookings, difficulty monitoring ticket sales, and inefficient check-in processes.

EventHub attempts to combine these operations into a single digital platform.

The application provides a mobile-first interface where users can discover events and perform ticket-related operations. Organizers are given tools to publish events and manage ticket inventory, while administrators can monitor the platform.

The application is built using Flutter, allowing the same codebase to target multiple platforms. Supabase provides backend infrastructure including authentication, PostgreSQL database services, storage, and server-side functionality.

The project architecture separates the presentation layer, state-management layer, and data-access layer. This makes the application easier to maintain and provides a foundation for future expansion.

---

# 3. Problem Statement

Traditional event management systems often have several limitations:

1. Manual event registration can be time-consuming.
2. Ticket availability may not be updated immediately.
3. Manual ticket verification can cause delays at event entrances.
4. Duplicate ticket usage can be difficult to detect.
5. Organizers may not have real-time visibility into ticket sales.
6. Event information may be distributed across multiple platforms.
7. Managing different ticket categories manually can become complicated.

8. Attendance and revenue reports may require manual preparation.

The problem addressed by EventHub is:

> To design and develop a centralized digital platform that simplifies event creation, ticket booking, ticket management, QR-based verification, attendance tracking, and event analytics.

---

# 4. Aim of the Project

The main aim of EventHub is to develop a cross-platform event management and ticket booking system that connects event organizers, attendees, and administrators through a unified application.

The system aims to make the complete event lifecycle digital:

**Event Creation → Event Publishing → Ticket Discovery → Booking → Payment → Digital Ticket → QR Verification → Attendance → Analytics**

---

# 5. Objectives

The major objectives of the project are:

## 5.1 Event Management

To allow organizers to create, edit, publish, and manage events.

## 5.2 Ticket Management

To allow organizers to define multiple ticket categories such as:

- General
- Early Bird
- VIP
- Other custom ticket tiers

Each ticket category can have its own price and inventory.

### 5.3 Online Ticket Booking

To allow attendees to browse events and book tickets digitally.

### 5.4 Secure Authentication

To implement authentication using Supabase Auth and provide different access levels according to user roles.

### 5.5 QR-Based Ticketing

To generate digital QR tickets that can be presented at the venue.

### 5.6 QR-Based Check-In

To allow event staff or organizers to scan attendee QR codes and validate tickets.

### 5.7 Inventory Protection

To prevent multiple users from purchasing the same remaining ticket when bookings occur simultaneously.

### 5.8 Analytics

To provide organizers with information such as:

- Tickets sold
- Revenue
- Attendance
- Attendance rate
- Ticket sell-through

### 5.9 Administrative Management

To provide administrators with platform-level monitoring and management capabilities.

---

# 6. Scope of the Project

The project covers three major categories of users.

## 6.1 Attendee

An attendee can:

- Register/login
- Browse events
- Search events
- Filter events
- View event details
- View ticket types
- Select tickets
- Proceed to checkout
- Apply promotional codes
- View bookings
- Access digital QR tickets
- Request refunds
- Participate in waitlists
- Transfer tickets where supported
- Submit event reviews where supported

## 6.2 Organizer

An organizer can:

- Create events
- Add event descriptions
- Add venue information
- Upload event banners
- Define ticket tiers
- Manage ticket inventory
- Monitor attendees
- View event analytics
- Manage promotional codes
- Scan attendee QR tickets
- View attendance information

## 6.3 Administrator

An administrator is responsible for platform-level operations such as:

- Monitoring users
- Managing roles
- Reviewing events

- Approving or rejecting events
- Viewing platform statistics
- Monitoring bookings
- Monitoring revenue
- Managing platform categories and cities

---

# 7. Existing System

In a conventional event management workflow, different activities may be performed using different tools.

For example:

- Event information may be shared using social media.
- Registration may happen through Google Forms.
- Payments may be handled separately.
- Tickets may be generated manually.
- Attendance may be recorded on paper.
- Revenue may be calculated using spreadsheets.

This approach creates several problems:

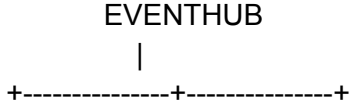
- Data duplication
- Manual errors
- Poor synchronization
- Difficult reporting
- Slow ticket verification
- Higher administrative workload

---

# 8. Proposed System

EventHub combines these operations into a centralized application.

The proposed workflow is:



```
    |           |           |
  ATTENDEE    ORGANIZER     ADMIN
    |           |           |
Browse Events  Create Event  Monitor Platform
    |           |           |
Select Ticket  Add Ticket Tiers  Review Events
    |           |           |
Checkout       Track Sales    Manage Users
    |           |           |
Booking        Scan Tickets   View Statistics
    |
QR Ticket
```

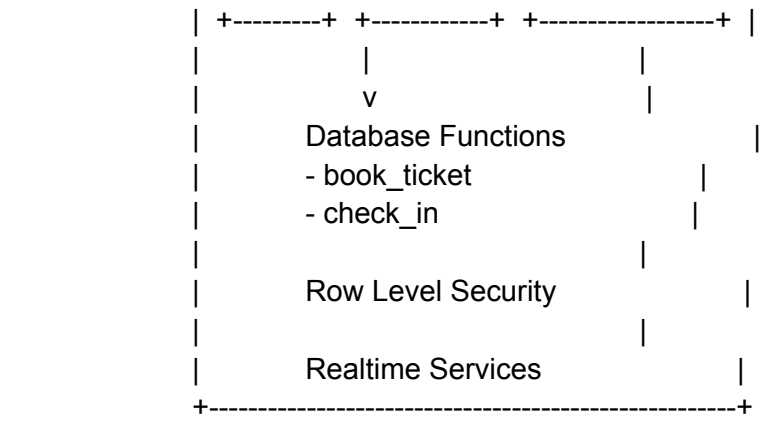
The centralized architecture allows the different components of the event lifecycle to communicate through the same backend.

---

# 9. System Architecture

EventHub follows a layered application architecture.

```
+-----------------------------------------------------+
|               FLUTTER APPLICATION                   |
|                                                     |
|  +-------------+  +-------------+  +--------------+ |
|  | Attendee UI |  | Organizer UI|  |   Admin UI   | |
|  +-------------+  +-------------+  +--------------+ |
|             |                            |          |
|             v                            |          |
|       Riverpod State Management                     |
|             |                            |          |
|             v                            |          |
|         Repository Layer                            |
+-----------------|-----------------------------------+
                  |
                  | HTTPS
                  v
+-----------------------------------------------------+
|                 SUPABASE                            |
|                                                     |
|  +---------+  +------------+  +------------------+  |
|  |  Auth   |  | PostgreSQL |  |     Storage      |  |
```




The repository documentation explicitly describes this separation into UI, Riverpod providers, repositories, and Supabase backend services.

---

# 10. Technology Stack

## 10.1 Flutter

Flutter is used to develop the application interface.

Advantages:

- Cross-platform development
- Single codebase
- Widget-based UI
- Hot reload
- Strong Dart typing
- Native compilation

The project configuration uses Flutter/Dart and includes packages specifically designed for Flutter application development.

---

## 10.2 Dart

Dart is the programming language used by Flutter.

It provides:

- Strong typing
- Object-oriented programming
- Null safety
- Asynchronous programming
- Future and Stream support

---

## 10.3 Supabase

Supabase acts as the backend platform.

The project uses Supabase for:

- Authentication
- PostgreSQL database
- Database functions
- Storage
- Realtime functionality
- Backend integration

---

## 10.4 PostgreSQL

PostgreSQL is the relational database used by Supabase.

It is particularly useful for this project because ticket booking requires transactional consistency.

For example, when only one ticket remains, two users should not be able to successfully purchase it simultaneously.

---

## 10.5 Riverpod

Riverpod is used for state management.

The application uses generated providers to connect repositories with UI components.

The general flow is:

Supabase
↓

Repository
↓
Riverpod Provider
↓
Flutter UI

---

## 10.6 go_router

go_router is used for application navigation.

The repository defines routes such as:

/login

/attendee/browse

/attendee/event/:id

/attendee/event/:id/seats

/attendee/event/:id/checkout

/attendee/my_tickets

/organizer/dashboard

/organizer/create_event

/organizer/event/:id/tickets

/organizer/event/:id/analytics

/organizer/event/:id/promo

/organizer/event/:id/attendees

/admin/dashboard

The router also performs role-based redirection.

---

## 10.7 Freezed

Freezed is used for immutable data models.

Models such as:

- AppUser
- Event
- Booking
- TicketType

are defined using Freezed annotations.

This reduces boilerplate and provides generated serialization and immutable model functionality.

---

## 10.8 QR Flutter

`qr_flutter` is used to display QR codes representing digital tickets.

---

## 10.9 Mobile Scanner

`mobile_scanner` provides camera-based QR scanning functionality for venue check-in.

---

## 10.10 fl_chart

`fl_chart` is used for displaying event analytics and graphical information.

---

## 10.11 Other Packages

The project also uses packages including:

- `cached_network_image`
- `intl`
- `flutter_dotenv`
- `google_fonts`
- `uuid`

- `url_launcher`
- `json_serializable`
- `build_runner`
- `riverpod_generator`

The dependency configuration is visible in the project's `pubspec.yaml`.

---

# 11. System Modules

## Module 1: Authentication

The authentication module handles:

- User registration
- Login
- Logout
- Authentication state
- User roles

The project defines three primary roles:

ATTENDEE
ORGANIZER
ADMIN

The `AppUser` model contains the user's ID, name, email, phone, role, FCM token, and creation timestamp.

---

# 12. Role-Based Navigation

After authentication, the application checks the user's role.

The routing logic works approximately as follows:

User Login
    |
    v

Authentication Successful
    |
    v
Read User Role
    |
    +---- Attendee ------> Attendee Browse
    |
    +---- Organizer -----> Organizer Dashboard
    |
    +---- Admin ---------> Admin Dashboard

The actual `go_router` implementation contains role-based redirects for these three roles.

---

# 13. Event Management Module

Organizers can create events containing information such as:

- Event title
- Description
- Venue
- City
- Start date/time
- End date/time
- Banner image
- Category
- Event status

The event model also contains:

- Total tickets sold
- Gross revenue
- Organizer ID
- Creation timestamp

The repository provides operations for retrieving published events, retrieving event details, retrieving organizer events, and creating events.

---

# 14. Ticket Tier Management

Each event can have multiple ticket types.

For example:

| Ticket Type | Price | Quantity |
|---|---|---|
| Early Bird | ₹500 | 100 |
| General | ₹800 | 300 |
| VIP | ₹1,500 | 50 |

Each ticket type contains:

- Ticket ID
- Event ID
- Name
- Price
- Total quantity
- Quantity sold
- Sales start time
- Sales end time

This allows organizers to create different pricing strategies for the same event.

---

# 15. Event Discovery

Attendees can browse published events.

The system can organize event discovery according to:

- Category
- City
- Date
- Event availability

The event repository retrieves published events and orders them by start time.

---

# 16. Event Details

When an attendee selects an event, the application can display:

- Event title
- Event description
- Venue
- City
- Date
- Time
- Banner
- Ticket types
- Ticket pricing
- Availability

The event details page acts as the main decision point before booking.

---

# 17. Ticket Booking Module

Ticket booking is one of the most important parts of the project.

A booking contains:

Booking ID
User ID
Event ID
Ticket Type ID
Quantity
Unit Price
Total Amount
Status
Idempotency Key
QR Token
Created At

The Flutter booking repository calls the Supabase `book_ticket` RPC instead of directly inserting a booking from the client.

This is an important architectural decision because inventory-sensitive operations should be handled transactionally on the backend.

# 18. Concurrency Control

A major problem in ticket booking systems is the race condition.

Consider an event with:

Available tickets = 1

Two users click "Book" at almost exactly the same time.

A poorly designed system might perform:

User A → Check availability → 1
User B → Check availability → 1

User A → Book
User B → Book

Result:

1 ticket available
2 successful bookings

This is called overselling.

The documented EventHub architecture addresses this using PostgreSQL row-level locking.

Conceptually:

```
SELECT quantity_total - quantity_sold
FROM ticket_types
WHERE id = ?
FOR UPDATE;
```

The row is locked while the transaction executes.

This means concurrent booking requests are serialized for the affected ticket tier.

The repository documentation also describes idempotency keys to prevent duplicate bookings caused by request retries.

# 19. Idempotency

An idempotency key is generated for each booking request.

Conceptually:

Generate UUID
↓
Send UUID with booking request
↓
Database checks existing booking
↓
If already exists → return existing booking
↓
Otherwise → create booking

This protects against a situation where a network failure occurs after the server processes the booking but before the client receives the response.

The booking repository generates a UUID and passes it to the `book_ticket` RPC.

---

# 20. Payment Module

The architecture documents support for payment processing through services such as Razorpay or Stripe.

The intended flow is:

User selects tickets
↓
Booking created
↓
Payment order/session
↓
User completes payment
↓
Payment provider webhook
↓
Server verifies payment
↓
Booking confirmed

↓
Digital ticket generated

However, this should be described carefully in an academic presentation.

The repository's VIVA documentation itself lists production Razorpay/Stripe integration among backend items still not started, while the application contains a simulated checkout experience. Therefore, the current project should not be presented as having a fully production-integrated payment gateway unless that integration has separately been completed.

---

# 21. QR Ticket System

After a successful booking, the system generates a QR-based digital ticket.

The QR ticket contains information associated with:

Event ID
Booking ID
QR Token

Conceptually:

EVT:<event_id>
BK:<booking_id>
T:<qr_token>

The attendee can display this QR code at the venue.

The QR is generated/displayed using the QR functionality included in the Flutter application.

---

# 22. QR Check-In System

At the venue:

Attendee
↓
Shows QR Ticket
↓
Organizer scans QR

↓
QR decoded
↓
Booking validated
↓
Check-in operation
↓
Booking marked as checked-in

The documented `check_in` RPC uses a conditional update based on the booking's current status.

For example:

confirmed → checked_in

If the ticket has already been checked in, the operation can return an invalid/already-used response.

This prevents a single booking from being accepted repeatedly.

---

# 23. Booking Status Lifecycle

The system supports several booking states.

pending_payment
|
v
confirmed
|
v
checked_in

Other possible states include:

cancelled
expired
refunded

This state-based approach makes the booking lifecycle easier to track.

---

# 24. Promo Code Module

The system also contains a promotional code mechanism.

An organizer can create a promo code with:

- Code
- Percentage discount
- Fixed discount
- Maximum uses
- Expiry date

At checkout:

Enter promo code
↓
Validate event
↓
Check expiration
↓
Check usage limit
↓
Calculate discount
↓
Display final price

The repository contains functionality for retrieving, validating, and creating promotional codes.

---

# 25. Refund Module

The project includes a refund request workflow.

An attendee can submit:

Booking ID
Refund Amount
Reason

The refund request initially receives:

pending

An organizer/admin workflow can subsequently change the status.

Possible statuses include:

pending
approved
rejected

The repository also updates the booking status to `refunded` when a refund is approved.

---

# 26. Ticket Transfer

The booking repository also contains a ticket transfer operation.

The intended workflow is:

Current Ticket Owner
↓
Enter recipient email
↓
Find recipient account
↓
Validate recipient
↓
Transfer booking ownership
↓
Generate new QR token

Generating a new QR token after transfer is important because the previous ticket credential should not remain valid as the primary ticket credential after ownership changes.

---

# 27. Reviews and Ratings

The event repository includes functionality for event reviews.

An attendee can submit:

- Rating
- Comment

- Event ID
- User information

The repository uses an upsert operation based on the event and user combination, allowing an attendee's review to be updated rather than creating unlimited duplicate reviews.

---

# 28. Analytics Module

Analytics are provided at the event and platform levels.

Event-level metrics include:

- Tickets sold
- Revenue
- Attendees
- Attendance rate

The documented SQL view calculates metrics from booking records.

Example conceptual calculation:

Attendance Rate =
Checked-in Tickets
---------- × 100
Confirmed Tickets

The project documentation also proposes visualizations such as:

- Revenue over time
- Tickets sold over time
- Sell-through by ticket tier
- Attendance rate
- Bookings by city/category

The Flutter dependency `fl_chart` is used for chart visualization.

---

# 29. Organizer Dashboard

The organizer dashboard is intended to provide a centralized view of event performance.

Typical information includes:

Total Tickets Sold
Total Revenue
Attendance
Attendance Rate
Ticket Tier Performance
Attendee List

Organizers can also access event-specific management screens for:

- Ticket tiers
- Analytics
- Promo codes
- Attendees

These routes are explicitly present in the application's router.

---

# 30. Admin Dashboard

The administrator module is designed for platform-level control.

The documented system includes:

- Event approval
- User management
- Role management
- Platform statistics
- Revenue monitoring
- Booking monitoring

The current project should distinguish between the **documented admin design** and the actual backend implementation available in the repository.

---

# 31. Database Design

The main conceptual entities are:

Users

Events
Ticket Types
Bookings
Payments
Check-ins
Promo Codes
Refunds
Reviews
Waitlist

Relationships can be represented as:

```
USER
 |
 +---- creates ----> EVENTS
 |
 +---- makes -----> BOOKINGS
                    |
                    +---- belongs to ----> EVENT
                    |
                    +---- uses ----------> TICKET TYPE
                    |
                    +---- generates -----> QR TICKET
                    |
                    +---- has -----------> PAYMENT
                    |
                    +---- produces ------> CHECK-IN
```

---

# 32. User Entity

The user model contains:

| Attribute | Description |
|---|---|
| ID | Unique user identifier |
| Name | User's name |
| Email | User email |
| Phone | Optional contact |
| Role | Attendee/Organizer/Admin |

| | |
|---|---|
| FCM Token | Notification token |
| Created At | Account creation time |

The Flutter `AppUser` model explicitly represents these fields.

---

# 33. Event Entity

The Event model contains:

| Attribute | Description |
|---|---|
| ID | Event identifier |
| Organizer ID | Owner/creator |
| Title | Event name |
| Description | Event information |
| Venue | Event location |
| City | Event city |
| Start At | Start date/time |
| End At | End date/time |
| Banner URL | Event image |
| Category | Event category |
| Status | Publication state |
| Total Sold | Tickets sold |
| Gross Revenue | Revenue |
| Created At | Creation timestamp |

---

# 34. Ticket Type Entity

The ticket type contains:

| Attribute | Description |
|---|---|
| ID | Ticket identifier |
| Event ID | Related event |
| Name | Ticket tier |
| Price | Ticket price |
| Quantity Total | Maximum inventory |
| Quantity Sold | Number sold |
| Sales Start | Beginning of sales |
| Sales End | End of sales |

---

# 35. Booking Entity

The booking entity contains:

| Attribute | Description |
|---|---|
| ID | Booking identifier |
| User ID | Customer |
| Event ID | Event |
| Ticket Type ID | Ticket category |
| Quantity | Number of tickets |
| Unit Price | Price per ticket |
| Total Amount | Total booking value |
| Status | Booking lifecycle state |

| | |
|---|---|
| Idempotency Key | Duplicate request protection |
| QR Token | Digital ticket credential |
| Created At | Booking timestamp |

The corresponding Freezed models are implemented in the Flutter project.

---

# 36. Data Flow

## Event Creation

Organizer
↓
Create Event Screen
↓
Riverpod Provider
↓
Event Repository
↓
Supabase
↓
PostgreSQL
↓
Event Created

## Ticket Booking

Attendee
↓
Event Details
↓
Select Ticket
↓
Checkout
↓
Booking Repository
↓
book_ticket RPC
↓

PostgreSQL Transaction
↓
Booking Created

## QR Check-In

Attendee QR
↓
Mobile Scanner
↓
Decode QR
↓
Booking Validation
↓
check_in RPC
↓
Database Update
↓
Checked In

---

# 37. State Management Architecture

The application follows a Riverpod-based reactive architecture.

The flow is:

Repository
↓
Provider
↓
AsyncValue
↓
ConsumerWidget
↓
UI

For example:

final events = ref.watch(publishedEventsProvider);

The UI then handles:

Loading

Error
Data

This prevents individual screens from directly managing all networking and state logic.

---

# 38. Navigation Architecture

The application uses declarative routing.

Authentication is integrated into the router.

The basic process is:

```
Router
  |
  +-- Is user authenticated?
        |
        +-- No → Login
        |
        +-- Yes
             |
             +-- Attendee → Browse
             |
             +-- Organizer → Dashboard
             |
             +-- Admin → Dashboard
```

This provides centralized navigation and role-based entry points.

---

# 39. UI and Theme

The application uses Material 3.

The theme includes:

- Primary purple color
- Secondary accent color
- Light theme
- Dark theme

- Google Inter font
- Rounded buttons
- Minimal AppBar elevation

The theme supports system-based light/dark mode selection.

---

# 40. Security

Security is an important part of the system architecture.

The documented design uses:

## Authentication

Supabase Auth manages user authentication.

## Authorization

User roles determine access.

## Row Level Security

Supabase/PostgreSQL RLS is intended to restrict data access according to authenticated users.

## Booking Protection

Database transactions protect ticket inventory.

## Idempotency

Unique idempotency keys reduce duplicate booking risks.

## QR Security

Random QR tokens are used as ticket credentials.

## Payment Verification

The documented production architecture expects payment confirmation through server-side webhooks rather than trusting a client-side success callback.

---

# 41. Environment Configuration

The project includes `.env.example` with configuration fields for:

SUPABASE_URL
SUPABASE_ANON_KEY
RAZORPAY_KEY_ID
STRIPE_PUBLISHABLE_KEY
APP_ENV

The project also uses `flutter_dotenv` to load environment configuration.

---

# 42. Important Implementation Observation

There is an important distinction between the project's intended architecture and its currently visible implementation.

The README describes advanced backend infrastructure including:

- Payment webhooks
- Booking expiry
- Full RLS
- Supabase Edge Functions
- Offline scanner synchronization
- Materialized analytics views
- Production payment processing

However, the VIVA guide later lists several of these backend components as not started.

Therefore, for academic evaluation, the safest description is:

> EventHub implements the core Flutter application architecture and provides the backend design for production-grade event booking. Some advanced backend and production-integration components remain part of the planned architecture or require completion/configuration.

This distinction prevents the report from claiming production functionality that has not been demonstrated.

---

# 43. Testing Strategy

The repository documentation proposes several levels of testing.

## Unit Testing

Individual functions and repositories can be tested independently.

Examples:

- Authentication logic
- Booking logic
- Validation
- Model serialization

## Widget Testing

Flutter widgets and screens can be tested independently.

## Integration Testing

The complete workflow can be tested:

Login
↓
Browse Event
↓
Select Ticket
↓
Checkout
↓
Booking
↓
QR Ticket

## Concurrency Testing

Concurrency testing is especially important for ticket booking.

The documented test scenario uses:

50 parallel booking attempts
10 available tickets

Expected result:

10 successful
40 SOLD_OUT

This is designed to validate the database-level inventory protection.

---

# 44. Deployment

The Flutter project can be built for multiple platforms.

### Android

flutter build appbundle --release

### iOS

flutter build ipa --release

### Web

flutter build web --release

The project documentation describes deployment to services such as Vercel, Netlify, or Firebase Hosting for the web build.

---

# 45. Continuous Integration

The repository contains GitHub Actions workflow infrastructure.

The documented CI pipeline performs:

Checkout repository
↓
Install Flutter

↓
flutter pub get
↓
flutter analyze
↓
flutter test

This helps detect compilation, static-analysis, and testing issues automatically.

---

# 46. Advantages of the System

## 46.1 Cross-Platform

One Flutter codebase can support multiple platforms.

## 46.2 Centralized Management

Events, tickets, bookings, and attendees can be managed from a single system.

## 46.3 Role-Based Architecture

Different users receive different functionality.

## 46.4 QR Ticketing

Digital tickets reduce dependence on physical tickets.

## 46.5 Transactional Booking

Database-level booking operations are designed to protect inventory.

## 46.6 Analytics

Organizers can understand event performance through numerical and graphical information.

## 46.7 Modular Codebase

The separation between models, repositories, providers, and features improves maintainability.

## 46.8 Expandability

The architecture provides room for additional features such as seat selection, recommendations, notifications, and third-party APIs.

---

# 47. Limitations

Despite its broad architecture, several limitations should be considered.

### 47.1 Production Payment Integration

The documented Razorpay/Stripe integration is not equivalent to a completed production payment system.

### 47.2 Backend Completion

Some advanced backend components described in the architecture still require implementation/configuration.

### 47.3 Offline Synchronization

Offline QR validation requires reliable local caching and synchronization logic. This should be tested thoroughly before being considered production-ready.

### 47.4 Security Policies

RLS must be fully implemented and tested rather than merely documented.

### 47.5 Scalability

Platform-wide statistics should use database aggregation rather than loading large datasets into the Flutter client.

### 47.6 Error Handling

Production applications should provide structured error types and user-friendly error messages instead of generic exceptions.

---

# 48. Future Enhancements

Possible future improvements include:

## Seat Selection

Users could select individual seats using a visual venue map.

## Group Bookings

Users could book tickets for multiple attendees.

## Multi-Currency

The platform could support currencies such as:

- INR
- USD
- EUR

## Organizer Payouts

Event organizers could receive automated payouts.

## Push Notifications

Notifications could be used for:

- Event reminders
- Booking confirmations
- Waitlist availability
- Event changes

## Calendar Integration

Users could add events directly to:

- Google Calendar
- Apple Calendar

## Public API

Third-party applications could integrate with EventHub.

## Recommendation Engine

Machine-learning models could recommend events based on user activity.

## White-Label Support

Organizations could deploy branded versions of the platform.

---

# 49. Feasibility

The project is technically feasible because it uses established technologies:

Flutter
+
Supabase
+
PostgreSQL
+
Riverpod
+
QR Technology

The architecture is also suitable for incremental development.

A basic version can operate with:

Authentication
+
Events
+
Ticket Types
+
Bookings
+
QR Tickets

Additional functionality can then be added without completely redesigning the application.

---

# 50. Applications of the System

The system can be adapted for:

- College festivals
- Technical events
- Workshops
- Seminars
- Conferences
- Music events
- Sports events
- Cultural programs
- Local community events
- Corporate events
- Hackathons
- Networking events

---

# 51. Learning Outcomes

Through this project, the following concepts can be demonstrated:

### Flutter

- Widgets
- Navigation
- State management
- Responsive UI
- Cross-platform development

### Dart

- Classes
- Async/await
- Futures
- Streams
- Null safety
- Generics

### Backend Development

- REST/database communication
- Authentication
- Authorization
- Database functions
- Cloud backend services

### Database

- Relational data modeling
- Foreign keys
- Transactions
- Concurrency control
- Aggregation
- Database security

### Software Engineering

- Modular architecture
- Repository pattern
- Separation of concerns
- Testing
- CI/CD
- Environment configuration

---

# 52. Conclusion

EventHub is a comprehensive Flutter-based event management and ticket booking application designed to digitalize the complete event lifecycle.

The system brings together event creation, ticket management, attendee booking, QR-based ticketing, venue check-in, analytics, and administrative monitoring into a single platform.

One of the most technically important aspects of the project is its approach to ticket inventory. Rather than depending entirely on client-side availability checks, the documented architecture moves the critical booking operation to PostgreSQL through an RPC function and uses transaction-level locking to address concurrent booking requests.

The use of Flutter, Riverpod, go_router, Freezed, Supabase, PostgreSQL, QR technology, and analytics libraries demonstrates a modern application development stack.

The project also provides a foundation for future improvements including real payment gateway integration, visual seat selection, push notifications, organizer payouts, recommendation systems, public APIs, and multi-tenant deployments.

From an academic perspective, EventHub demonstrates practical implementation of mobile application development, cloud backend integration, database management, authentication, authorization, state management, concurrency handling, and software architecture.

---

# 53. References


1. Flutter Documentation
2. Dart Documentation
3. Supabase Documentation
4. PostgreSQL Documentation
5. Riverpod Documentation
6. go_router Documentation
7. Freezed Documentation
8. fl_chart Documentation
9. qr_flutter Documentation
10. mobile_scanner Documentation
11. EventHub GitHub Repository
12. EventHub Project VIVA Guide


---

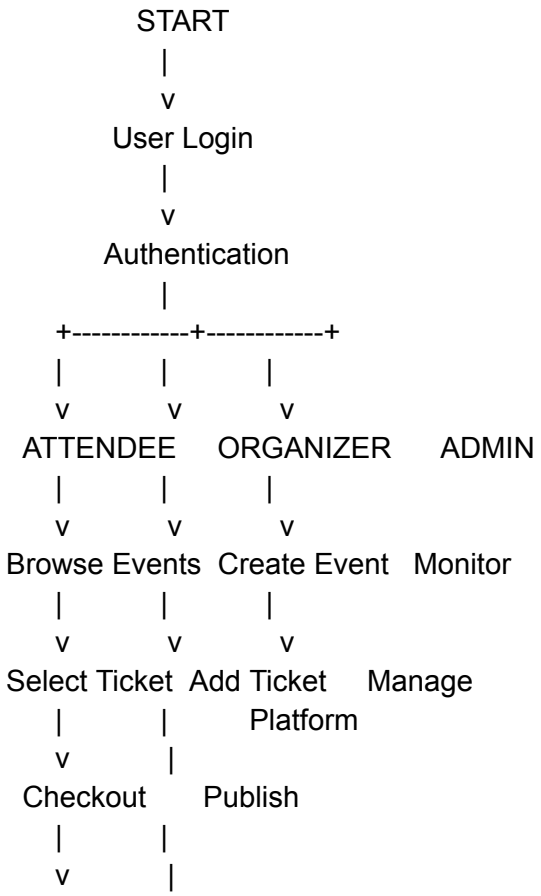
# 54. Project Summary

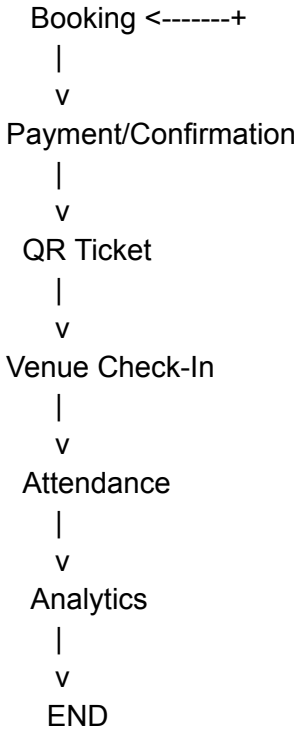
| Component | Technology |
|---|---|
| Frontend | Flutter |
| Programming Language | Dart |
| State Management | Riverpod |
| Navigation | go_router |
| Data Models | Freezed |
| Backend | Supabase |

| | |
|---|---|
| Database | PostgreSQL |
| Authentication | Supabase Auth |
| QR Generation | qr_flutter |
| QR Scanning | mobile_scanner |
| Analytics | fl_chart |
| Configuration | flutter_dotenv |
| Version Control | Git/GitHub |
| CI/CD | GitHub Actions |
| Architecture | Layered / Repository-based |

---

# 55. Overall System Workflow






# Final Statement

EventHub provides a strong foundation for a modern event-management platform by combining cross-platform Flutter development with a relational cloud backend. Its architecture is particularly suitable for demonstrating real-world concepts such as role-based access, transactional booking, QR ticket validation, analytics, and modular software design.